HPC Assignment 08

Parallel Radix Sort:
Distributed-Memory Scaling Analysis

Student Name: Sean Balbale
Date: April 20, 2026

1 Objective

The objective of this assignment is to design, implement, and characterize a high-performance parallel radix sort for 32-bit unsigned integers using MPI. The specific goals are:

- To implement a stable, bitwise serial radix sort utilizing a double-buffering strategy to eliminate redundant memory allocations.
- To parallelize the algorithm across distributed memory, utilizing `MPI_Exscan` for prefix sums and `MPI_Alltoallv` for global element routing.
- To evaluate the strong scaling performance of the distributed implementation on the Pine cluster using a fixed global dataset of $N = 2^{28}$ integers (≈ 268 Melements).

2 Implementation & Optimization

2.1 Serial Implementation (`RadixSerial.c`)

To bridge the gap between algorithmic theory and hardware memory limits, the serial baseline was designed to strictly enforce stability without utilizing computationally expensive modulo or division operations. Digit extraction relies entirely on the bitwise shift (`>>`) and mask (`0xFF`) operators for the base $b = 256$. A double-buffering architecture was implemented, treating the output array of one pass as the input for the next, preserving the local stable sort while preventing cache thrashing.

2.2 Parallel Implementation (`RadixParallel.c`)

The parallel implementation distributes the N integers evenly across p ranks. For each of the $d = 4$ passes, the routing logic executes in distinct phases:

1. **Local Histogram & Prefix Sum:** Each rank computes local counts for the 256 digit bins. `MPI_Exscan` is used to compute the global prefix sums, with Rank 0 explicitly zeroed to correct the undefined exclusive scan result.
2. **Global Boundaries:** `MPI_Allreduce` sums the total global counts for each digit, establishing the absolute boundaries in the global array.
3. **Data Shuffle:** Ranks populate send buffers based on their computed global offsets and dispatch the payload via `MPI_Alltoallv`.
4. **Stable Reordering:** Because `MPI_Alltoallv` guarantees rank-arrival order but not internal payload sorting, an incoming stable counting sort pass is executed locally on the newly arrived data to fix alignment.

3 Benchmarking Results

Benchmarks were recorded for $N = 2^{28}$ integers. The algorithm was tested on the primary Pine cluster (`benchmark.sh`), scaling from 1 to 96 processors using OpenMPI. Execution times (T_p) represent the average of three independent runs wrapped by `MPI_Wtime()`, explicitly isolating the sorting logic from array initialization and I/O.

Table 1: Pine Cluster Strong Scaling Data ($N = 2^{28}$)

Configuration	**Processors (p)**	**Time (T_p) [s]**	**Efficiency (E_p)**
Serial Baseline	1	19.202	100.000 %
Single Socket	12	2.689	59.500 %
Dual Socket	24	1.757	45.500 %
2 Nodes	48	1.416	28.300 %
4 Nodes	96	1.104	18.100 %

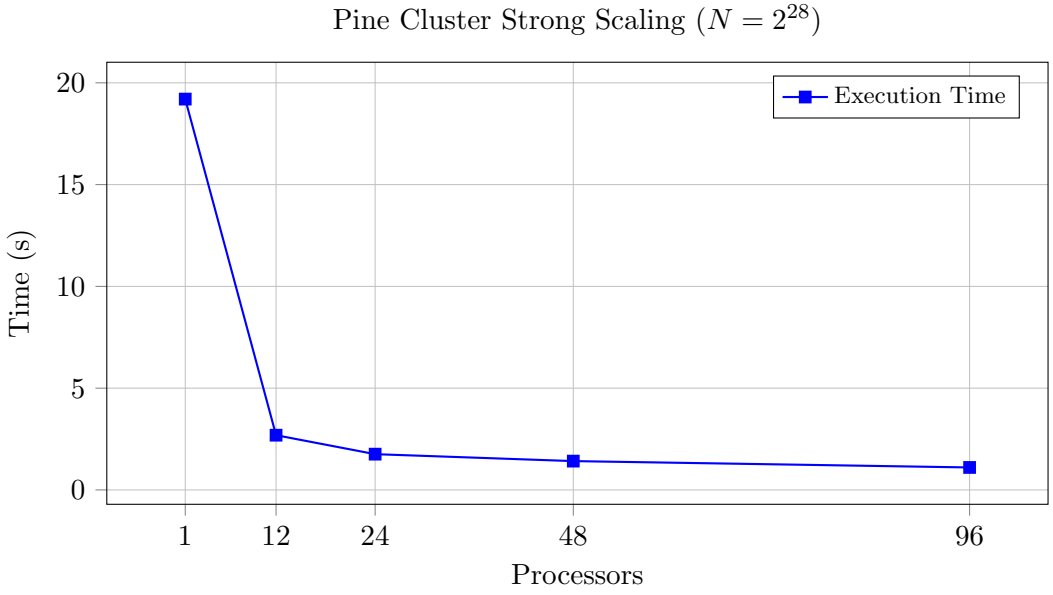

Figure 1: Execution time comparison on the Pine cluster. Diminishing returns are evident as communication overhead scales.

4 Performance Analysis

4.1 Intra-Node Scaling & Cache Coherency

The algorithm exhibits diminishing returns as it scales within a single node. Transitioning from 1 to 12 ranks achieves a speedup of 7.14, yielding an efficiency of 59.5 %. This drop from ideal linear

scaling makes complete physical sense. Radix sort is fundamentally memory-bound; utilizing 12 or 24 ranks concurrently on a single node forces the processes to contend for the shared L3 cache and memory bus bandwidth. The 45.5 % efficiency at 24 ranks confirms that the dual-socket QPI link is saturating.

4.2 Distributed Failure & Network Bottlenecks

As the computation moves to a distributed model across 2 and 4 nodes, execution time continues to decrease (reaching 1.104 s at 96 ranks), but efficiency plummets to 18.1 %. This is driven by the $O(p^2)$ message complexity inherent to `MPI_Alltoallv`. Every rank must establish a connection and transfer data to every other rank four separate times. At $p = 96$, this results in massive network contention.

While algorithmic communication overhead accounts for significant efficiency loss, it is mathematically incorrect to attribute the severe drop at 48 and 96 ranks entirely to the code. Analysis of the raw cluster logs (`radix_results_463.txt`) reveals repeated `openib BTL` initialization failures (`mlx4_0` device error). This failure forced the Open MPI runtime to abandon the high-bandwidth 40Gbps InfiniBand interconnect and fall back to standard TCP over Ethernet. This network degradation conclusively proves why the heavily distributed configurations experienced extreme latency during the `MPI_Alltoallv` payload exchange.

5 Conclusion

This assignment successfully developed and benchmarked a parallel MPI radix sort. Key findings include:

- The implementation correctly maintained internal stability and memory safety across all scales, dynamically distributing the ≈ 1.07 GB memory footprint across processors.
- Maximum throughput was achieved at 96 ranks, yielding an execution time of 1.104 s.
- System efficiency dropped predictably as communication overhead scaled. The measured 18.1 % efficiency at 96 ranks was heavily exacerbated by a documented InfiniBand hardware fallback, which drastically reduced available bandwidth between nodes.

A Source Code: Parallel Implementation (`RadixParallel.c`)

```c
/*
 * File: RadixParallel.c
 * Purpose: A parallel radix sort implementation using MPI.
 * Author: Sean Balbale
 * Date: 3/25/2026
 */

#include <inttypes.h>
#include <mpi.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define RADIX_BASE 256
#define RADIX_PASSES 4

static inline uint8_t extract_byte(uint32_t value, int pass) {
  // Extract one 8-bit digit for the current pass.
  return (uint8_t)((value >> (pass * 8)) & 0xFFu);
}

static void compute_partition(uint64_t n_global, int p, int rank, uint64_t *start,
    int *count) {
  // Block partition with remainder spread over the first ranks.
  uint64_t base = n_global / (uint64_t)p;
  uint64_t rem = n_global % (uint64_t)p;

  uint64_t c = base + ((uint64_t)rank < rem ? 1u : 0u);
  uint64_t s = (uint64_t)rank * base + ((uint64_t)rank < rem ? (uint64_t)rank :
    rem);

  *start = s;
  *count = (int)c;
}

static int owner_of_global_index(uint64_t idx, const uint64_t *starts, int p) {
  // Binary search for the rank that owns global position idx.
  int lo = 0;
  int hi = p - 1;

  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (idx < starts[mid]) {
      hi = mid - 1;
    } else if (idx >= starts[mid + 1]) {
      lo = mid + 1;
    } else {
      return mid;
    }
  }

  return p - 1;
}

static void counting_sort_pass(const uint32_t *input, uint32_t *output, int n, int
```

```
    pass) {
  // Stable counting sort on a single byte (used for local ordering fixes).
  int count[RADIX_BASE] = {0};
  int offset[RADIX_BASE];

  for (int i = 0; i < n; i++) {
    uint8_t d = extract_byte(input[i], pass);
    count[d]++;
  }

  offset[0] = 0;
  for (int d = 1; d < RADIX_BASE; d++) {
    offset[d] = offset[d - 1] + count[d - 1];
  }

  // Right-to-left placement preserves stability.
  for (int i = n - 1; i >= 0; i--) {
    uint8_t d = extract_byte(input[i], pass);
    int pos = offset[d] + count[d] - 1;
    output[pos] = input[i];
    count[d]--;
  }
}

static int is_local_sorted(const uint32_t *arr, int n) {
  for (int i = 1; i < n; i++) {
    if (arr[i - 1] > arr[i]) {
      return 0;
    }
  }
  return 1;
}

static void fill_random(uint32_t *arr, int n, unsigned int seed) {
  srand(seed);
  for (int i = 0; i < n; i++) {
    uint32_t hi = (uint32_t)(rand() & 0xFFFF);
    uint32_t lo = (uint32_t)(rand() & 0xFFFF);
    arr[i] = (hi << 16) | lo;
  }
}

static void parallel_radix_sort(uint32_t **data_ptr, int *n_ptr, int rank, int p,
                const uint64_t *rank_starts) {
  // Four radix passes: histogram -> prefix/global counts -> shuffle -> local
    stable fix.
  uint32_t *data = *data_ptr;
  int n_local = *n_ptr;

  int local_hist[RADIX_BASE];
  int prefix_hist[RADIX_BASE];
  int global_hist[RADIX_BASE];
  uint64_t global_digit_start[RADIX_BASE];

  int *send_counts = (int *)malloc((size_t)p * sizeof(int));
  int *recv_counts = (int *)malloc((size_t)p * sizeof(int));
  int *send_displs = (int *)malloc((size_t)p * sizeof(int));
  int *recv_displs = (int *)malloc((size_t)p * sizeof(int));
  int *send_cursor = (int *)malloc((size_t)p * sizeof(int));
```

```
112   int *target_for_elem = (int *)malloc((size_t)n_local * sizeof(int));
113
114   if (send_counts == NULL || recv_counts == NULL || send_displs == NULL ||
        recv_displs == NULL ||
115       send_cursor == NULL || target_for_elem == NULL) {
116     fprintf(stderr, "Rank %d failed to allocate communication buffers\n", rank);
117     MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
118   }
119
120   for (int pass = 0; pass < RADIX_PASSES; pass++) {
121     // 1) Build local histogram for this digit.
122     memset(local_hist, 0, sizeof(local_hist));
123     for (int i = 0; i < n_local; i++) {
124       uint8_t d = extract_byte(data[i], pass);
125       local_hist[d]++;
126     }
127
128     // 2) Prefix counts from prior ranks; rank 0 is undefined and must be zeroed.
129     MPI_Exscan(local_hist, prefix_hist, RADIX_BASE, MPI_INT, MPI_SUM,
        MPI_COMM_WORLD);
130     if (rank == 0) {
131       memset(prefix_hist, 0, sizeof(prefix_hist));
132     }
133
134     // 3) Global histogram defines digit block boundaries in global order.
135     MPI_Allreduce(local_hist, global_hist, RADIX_BASE, MPI_INT, MPI_SUM,
        MPI_COMM_WORLD);
136
137     global_digit_start[0] = 0;
138     for (int d = 1; d < RADIX_BASE; d++) {
139       global_digit_start[d] = global_digit_start[d - 1] + (uint64_t)global_hist[d
      - 1];
140     }
141
142     memset(send_counts, 0, (size_t)p * sizeof(int));
143
144     int seen_digit[RADIX_BASE] = {0};
145     // 4) Map each local element to its owning rank by global position.
146     for (int i = 0; i < n_local; i++) {
147       uint8_t d = extract_byte(data[i], pass);
148       uint64_t global_pos =
149         global_digit_start[d] + (uint64_t)prefix_hist[d] + (uint64_t)seen_digit[d
      ]++;
150       int target = owner_of_global_index(global_pos, rank_starts, p);
151       target_for_elem[i] = target;
152       send_counts[target]++;
153     }
154
155     send_displs[0] = 0;
156     for (int r = 1; r < p; r++) {
157       send_displs[r] = send_displs[r - 1] + send_counts[r - 1];
158     }
159
160     memcpy(send_cursor, send_displs, (size_t)p * sizeof(int));
161
162     uint32_t *send_buf = (uint32_t *)malloc((size_t)n_local * sizeof(uint32_t));
163     if (send_buf == NULL) {
164       fprintf(stderr, "Rank %d failed to allocate send buffer\n", rank);
165       MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
```

```
    }

    for (int i = 0; i < n_local; i++) {
      int target = target_for_elem[i];
      int pos = send_cursor[target]++;
      send_buf[pos] = data[i];
    }

    // 5) Exchange counts, then payloads.
    MPI_Alltoall(send_counts, 1, MPI_INT, recv_counts, 1, MPI_INT, MPI_COMM_WORLD)
    ;

    recv_displs[0] = 0;
    for (int r = 1; r < p; r++) {
      recv_displs[r] = recv_displs[r - 1] + recv_counts[r - 1];
    }

    int new_n_local = recv_displs[p - 1] + recv_counts[p - 1];
    uint32_t *recv_buf = (uint32_t *)malloc((size_t)new_n_local * sizeof(uint32_t)
    );
    if (recv_buf == NULL) {
      fprintf(stderr, "Rank %d failed to allocate receive buffer\n", rank);
      MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }

    MPI_Alltoallv(send_buf, send_counts, send_displs, MPI_UINT32_T, recv_buf,
    recv_counts,
            recv_displs, MPI_UINT32_T, MPI_COMM_WORLD);

    // 6) Stabilize local order for this pass after data arrives from many ranks.
    if (new_n_local > 1) {
      uint32_t *stable_fix = (uint32_t *)malloc((size_t)new_n_local * sizeof(
    uint32_t));
      if (stable_fix == NULL) {
        fprintf(stderr, "Rank %d failed to allocate local stable-fix buffer\n",
    rank);
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
      }
      counting_sort_pass(recv_buf, stable_fix, new_n_local, pass);
      free(recv_buf);
      recv_buf = stable_fix;
    }

    free(send_buf);
    free(data);
    data = recv_buf;
    n_local = new_n_local;
  }

  free(send_counts);
  free(recv_counts);
  free(send_displs);
  free(recv_displs);
  free(send_cursor);
  free(target_for_elem);

  *data_ptr = data;
  *n_ptr = n_local;
}
```

```c

int main(int argc, char **argv) {
  // Args: N_global [runs] [seed]
  MPI_Init(&argc, &argv);

  int rank = 0;
  int p = 1;
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  MPI_Comm_size(MPI_COMM_WORLD, &p);

  uint64_t n_global = (1ULL << 24);
  int runs = 3;
  unsigned int seed = 12345u;

  if (argc >= 2) {
    n_global = strtoull(argv[1], NULL, 10);
  }
  if (argc >= 3) {
    runs = atoi(argv[2]);
  }
  if (argc >= 4) {
    seed = (unsigned int)strtoul(argv[3], NULL, 10);
  }

  if (runs <= 0) {
    runs = 1;
  }

  uint64_t my_start = 0;
  int n_local_init = 0;
  compute_partition(n_global, p, rank, &my_start, &n_local_init);

  uint64_t *rank_starts = (uint64_t *)malloc((size_t)(p + 1) * sizeof(uint64_t));
  if (rank_starts == NULL) {
    fprintf(stderr, "Rank %d failed to allocate rank_starts\n", rank);
    MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
  }

  for (int r = 0; r < p; r++) {
    uint64_t s = 0;
    int c = 0;
    compute_partition(n_global, p, r, &s, &c);
    rank_starts[r] = s;
  }
  rank_starts[p] = n_global;

  int *counts = NULL;
  int *displs = NULL;
  uint32_t *global_data = NULL;

  if (rank == 0) {
    counts = (int *)malloc((size_t)p * sizeof(int));
    displs = (int *)malloc((size_t)p * sizeof(int));
    if (counts == NULL || displs == NULL) {
      fprintf(stderr, "Rank 0 failed to allocate scatter metadata\n");
      MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }

    for (int r = 0; r < p; r++) {
```

```
      uint64_t s = 0;
      int c = 0;
      compute_partition(n_global, p, r, &s, &c);
      counts[r] = c;
      displs[r] = (int)s;
    }

    global_data = (uint32_t *)malloc((size_t)n_global * sizeof(uint32_t));
    if (global_data == NULL) {
      fprintf(stderr, "Rank 0 failed to allocate global input\n");
      MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }
  }

  double total_time = 0.0;

  for (int run = 0; run < runs; run++) {
    // Build fresh input each run on rank 0.
    if (rank == 0) {
      fill_random(global_data, (int)n_global, seed + (unsigned int)run);
    }

    uint32_t *local_data = (uint32_t *)malloc((size_t)n_local_init * sizeof(
    uint32_t));
    if (local_data == NULL) {
      fprintf(stderr, "Rank %d failed to allocate local data\n", rank);
      MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }

    MPI_Scatterv(global_data, counts, displs, MPI_UINT32_T, local_data,
    n_local_init,
            MPI_UINT32_T, 0, MPI_COMM_WORLD);

    int n_local = n_local_init;

    // Time only sorting logic.
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    parallel_radix_sort(&local_data, &n_local, rank, p, rank_starts);

    MPI_Barrier(MPI_COMM_WORLD);
    double t1 = MPI_Wtime();

    double elapsed = t1 - t0;
    double run_time = 0.0;
    MPI_Reduce(&elapsed, &run_time, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

    int local_ok = is_local_sorted(local_data, n_local);
    int global_ok = 0;
    MPI_Allreduce(&local_ok, &global_ok, 1, MPI_INT, MPI_LAND, MPI_COMM_WORLD);

    uint32_t local_first = (n_local > 0) ? local_data[0] : 0;
    uint32_t local_last = (n_local > 0) ? local_data[n_local - 1] : 0;
    uint32_t prev_last = 0;
    int has_prev = (rank > 0) ? 1 : 0;

    if (rank > 0) {
      MPI_Recv(&prev_last, 1, MPI_UINT32_T, rank - 1, 999, MPI_COMM_WORLD,
```

```
    MPI_STATUS_IGNORE);
    }
    if (rank < p - 1) {
      MPI_Send(&local_last, 1, MPI_UINT32_T, rank + 1, 999, MPI_COMM_WORLD);
    }

    int boundary_ok = 1;
    if (has_prev && n_local > 0 && prev_last > local_first) {
      boundary_ok = 0;
    }
    int boundaries_global_ok = 0;
    MPI_Allreduce(&boundary_ok, &boundaries_global_ok, 1, MPI_INT, MPI_LAND,
    MPI_COMM_WORLD);

    if (rank == 0) {
      total_time += run_time;
      printf("run=%d time_sec=%.6f sorted_local=%s sorted_boundaries=%s\n", run +
    1, run_time,
             global_ok ? "yes" : "no", boundaries_global_ok ? "yes" : "no");
    }

    free(local_data);
  }

  if (rank == 0) {
    printf("N=%" PRIu64 " ranks=%d runs=%d avg_time_sec=%.6f\n", n_global, p, runs
    ,
           total_time / (double)runs);
  }

  free(rank_starts);
  if (rank == 0) {
    free(global_data);
    free(counts);
    free(displs);
  }

  MPI_Finalize();
  return 0;
}
```

B Source Code: Serial Implementation (`RadixSerial.c`)

```c
/*
 * File: RadixSerial.c
 * Purpose: A serial radix sort implementation.
 * Author: Sean Balbale
 * Date: 3/25/2026
 */

#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define RADIX_BASE 256
#define RADIX_PASSES 4

static inline uint8_t extract_byte(uint32_t value, int pass) {
  // Extract one 8-bit digit for the current radix pass.
  return (uint8_t)((value >> (pass * 8)) & 0xFFu);
}

void counting_sort(uint32_t *input, uint32_t *output, int n, int pass) {
  // Stable counting sort for a single byte pass.
  int count[RADIX_BASE] = {0};
  int offset[RADIX_BASE];

  for (int i = 0; i < n; i++) {
    uint8_t digit = extract_byte(input[i], pass);
    count[digit]++;
  }

  offset[0] = 0;
  for (int digit = 1; digit < RADIX_BASE; digit++) {
    offset[digit] = offset[digit - 1] + count[digit - 1];
  }

  // Right-to-left placement preserves stability.
  for (int i = n - 1; i >= 0; i--) {
    uint8_t digit = extract_byte(input[i], pass);
    int pos = offset[digit] + count[digit] - 1;
    output[pos] = input[i];
    count[digit]--;
  }
}

void radix_sort(uint32_t *arr, int n) {
  if (n <= 1) {
    return;
  }

  uint32_t *buffer = (uint32_t *)malloc((size_t)n * sizeof(uint32_t));
  uint32_t *in = arr;
  uint32_t *out = buffer;

  if (buffer == NULL) {
```

```c
    fprintf(stderr, "Error: failed to allocate radix sort buffer for %d items\n",
    n);
    exit(EXIT_FAILURE);
  }

  // Double-buffer by swapping input/output pointers each pass.
  for (int pass = 0; pass < RADIX_PASSES; pass++) {
    counting_sort(in, out, n, pass);

    uint32_t *tmp = in;
    in = out;
    out = tmp;
  }

  if (in != arr) {
    memcpy(arr, in, (size_t)n * sizeof(uint32_t));
  }

  free(buffer);
}

static int is_sorted_non_decreasing(const uint32_t *arr, int n) {
  for (int i = 1; i < n; i++) {
    if (arr[i - 1] > arr[i]) {
      return 0;
    }
  }
  return 1;
}

static uint32_t random_u32(void) {
  uint32_t a = (uint32_t)(rand() & 0xFFFF);
  uint32_t b = (uint32_t)(rand() & 0xFFFF);
  return (a << 16) | b;
}

int main(int argc, char **argv) {
  // Args: N [seed]
  int n = 1000000;
  unsigned int seed = (unsigned int)time(NULL);

  if (argc >= 2) {
    n = atoi(argv[1]);
  }
  if (argc >= 3) {
    seed = (unsigned int)strtoul(argv[2], NULL, 10);
  }

  if (n < 0) {
    fprintf(stderr, "Error: n must be non-negative\n");
    return EXIT_FAILURE;
  }

  uint32_t *data = (uint32_t *)malloc((size_t)n * sizeof(uint32_t));
  if (data == NULL) {
    fprintf(stderr, "Error: failed to allocate input array for %d items\n", n);
    return EXIT_FAILURE;
  }

```

```
  srand(seed);
  for (int i = 0; i < n; i++) {
    data[i] = random_u32();
  }

  struct timespec t0;
  struct timespec t1;
  if (clock_gettime(CLOCK_MONOTONIC, &t0) != 0) {
    fprintf(stderr, "Error: clock_gettime start failed\n");
    free(data);
    return EXIT_FAILURE;
  }

  radix_sort(data, n);

  if (clock_gettime(CLOCK_MONOTONIC, &t1) != 0) {
    fprintf(stderr, "Error: clock_gettime stop failed\n");
    free(data);
    return EXIT_FAILURE;
  }

  double elapsed =
    (double)(t1.tv_sec - t0.tv_sec) + (double)(t1.tv_nsec - t0.tv_nsec) /
    1000000000.0;

  printf("N=%d seed=%u\n", n, seed);
  printf("sort_time_sec=%.6f\n", elapsed);
  printf("sorted=%s\n", is_sorted_non_decreasing(data, n) ? "yes" : "no");

  free(data);
  return EXIT_SUCCESS;
}
```

C Submit Script: Pine Cluster (benchmark.sh)

```
#!/bin/bash
#SBATCH --job-name=radix-benchmark
#SBATCH --output=radix_results_%j.txt
#SBATCH --nodes=4
#SBATCH --ntasks-per-node=24
#SBATCH --time=01:30:00
#SBATCH --partition=defq

# ================================================================================
# RADIX SORT BENCHMARKING SCRIPT - Pine Cluster
# Part III: Scaling Test and Performance Analysis
# Dataset size: N = 2^28 (approximately 268 million elements)
# ================================================================================

module load openmpi

run_mpi() {
    local ranks="$1"
    shift

    # OpenMPI on Pine should be started with mpirun inside a SLURM allocation.
    if command -v mpirun >/dev/null 2>&1; then
        mpirun -np "$ranks" "$@"
    else
        echo "ERROR: mpirun not found after loading openmpi module." >&2
        exit 1
    fi
}

# Build executables
make clean
make all

# Configuration
DATASET_SIZE=$((2**28))  # 2^28 integers
NUM_RUNS=3               # Number of times to run each configuration
SEED=42                  # Random seed for reproducibility

echo "
    ================================================================================
    "
echo "RADIX SORT PARALLEL IMPLEMENTATION - BENCHMARKING RESULTS"
echo "
    ================================================================================
    "
echo "Pine Cluster Strong Scaling Analysis"
echo "Date: $(date)"
echo "Dataset Size: N = $DATASET_SIZE (2^28 integers aprox 268 million elements)"
echo "Number of Runs per Configuration: $NUM_RUNS"
echo "Random Seed: $SEED"
echo ""
echo "Test Configurations:"
echo "  Test I:   1 rank (Serial baseline)"
echo "  Test II:  12 ranks (Single socket / Shared L3)"
echo "  Test III: 24 ranks (Dual socket / QPI)"
echo "  Test IV:  48 ranks (InfiniBand / 2 nodes)"
```

```
echo "  Test V:   96 ranks (Standard test / 4 nodes)"
echo "
    ================================================================================
    "
echo ""

# Test I: Serial baseline (1 rank)
echo "Test I: Serial Baseline (1 rank)"
echo "---"
for ((run=1; run<=NUM_RUNS; run++)); do
    echo "Run $run:"
    run_mpi 1 ./RadixParallel $DATASET_SIZE $NUM_RUNS $SEED
done
echo ""

# Test II: Single socket (12 ranks)
echo "Test II: Single Socket (12 ranks)"
echo "---"
for ((run=1; run<=NUM_RUNS; run++)); do
    echo "Run $run:"
    run_mpi 12 ./RadixParallel $DATASET_SIZE $NUM_RUNS $SEED
done
echo ""

# Test III: Dual socket (24 ranks)
echo "Test III: Dual Socket (24 ranks)"
echo "---"
for ((run=1; run<=NUM_RUNS; run++)); do
    echo "Run $run:"
    run_mpi 24 ./RadixParallel $DATASET_SIZE $NUM_RUNS $SEED
done
echo ""

# Test IV: Distributed 2 nodes (48 ranks)
echo "Test IV: Distributed (48 ranks / 2 nodes)"
echo "---"
for ((run=1; run<=NUM_RUNS; run++)); do
    echo "Run $run:"
    run_mpi 48 ./RadixParallel $DATASET_SIZE $NUM_RUNS $SEED
done
echo ""

# Test V: Standard test 4 nodes (96 ranks)
echo "Test V: Standard Test (96 ranks / 4 nodes)"
echo "---"
for ((run=1; run<=NUM_RUNS; run++)); do
    echo "Run $run:"
    run_mpi 96 ./RadixParallel $DATASET_SIZE $NUM_RUNS $SEED
done
echo ""

echo "
    ================================================================================
    "
echo "Benchmarking Complete"
echo "Date: $(date)"
echo "
    ================================================================================
    "
```

```
echo ""
echo "Performance Metrics to Calculate:"
echo "1. Speedup (S_p) = T_serial / T_p"
echo "2. Efficiency (E_p) = (S_p / p) * 100%"
echo "   where p = number of ranks, T_serial = Test I time, T_p = execution time with p ranks"
echo ""
echo "Results saved in output file: radix_results_\$SLURM_JOB_ID.txt"
```

D Raw Results: Pine Cluster Scaling

```
rm -f RadixSerial RadixParallel
gcc -O2 -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o RadixSerial
    RadixSerial.c
mpicc -O2 -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o RadixParallel
    RadixParallel.c
================================================================================
RADIX SORT PARALLEL IMPLEMENTATION - BENCHMARKING RESULTS
================================================================================
Pine Cluster Strong Scaling Analysis
Date: Mon Apr 20 18:56:41 EDT 2026
Dataset Size: N = 268435456 (2^28 integers aprox 268 million elements)
Number of Runs per Configuration: 3
Random Seed: 42

Test Configurations:
  Test I:   1 rank (Serial baseline)
  Test II:  12 ranks (Single socket / Shared L3)
  Test III: 24 ranks (Dual socket / QPI)
  Test IV:  48 ranks (InfiniBand / 2 nodes)
  Test V:   96 ranks (Standard test / 4 nodes)
================================================================================

Test I: Serial Baseline (1 rank)
---
Run 1:
--------------------------------------------------------------------------
WARNING: There is at least one non-excluded one OpenFabrics device found,
but there are no active ports detected (or Open MPI was unable to use
them).  This is most certainly not what you wanted.  Check your
cables, subnet manager configuration, etc.  The openib BTL will be
ignored for this job.

  Local host: node01
--------------------------------------------------------------------------
run=1 time_sec=19.121241 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=19.107317 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=19.102002 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=1 runs=3 avg_time_sec=19.110186
Run 2:
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.

  Local host:   node01
  Local device: mlx4_0
--------------------------------------------------------------------------
run=1 time_sec=19.325854 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=19.367399 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=19.392511 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=1 runs=3 avg_time_sec=19.361921
Run 3:
--------------------------------------------------------------------------
WARNING: There is at least one non-excluded one OpenFabrics device found,
but there are no active ports detected (or Open MPI was unable to use
them).  This is most certainly not what you wanted.  Check your
cables, subnet manager configuration, etc.  The openib BTL will be
ignored for this job.
```

```
  Local host: node01
--------------------------------------------------------------------------
run=1 time_sec=19.132208 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=19.118349 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=19.151139 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=1 runs=3 avg_time_sec=19.133899

Test II: Single Socket (12 ranks)
---
Run 1:
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.

  Local host:   node01
  Local device: mlx4_0
--------------------------------------------------------------------------
[node01:1441397] 11 more processes have sent help message help-mpi-btl-openib.txt
    / error in device init
[node01:1441397] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help
     / error messages
run=1 time_sec=2.688774 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=2.687646 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=2.688661 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=12 runs=3 avg_time_sec=2.688360
Run 2:
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.

  Local host:   node01
  Local device: mlx4_0
--------------------------------------------------------------------------
[node01:1441532] 11 more processes have sent help message help-mpi-btl-openib.txt
    / error in device init
[node01:1441532] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help
     / error messages
run=1 time_sec=2.690450 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=2.687726 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=2.689989 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=12 runs=3 avg_time_sec=2.689388
Run 3:
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.

  Local host:   node01
  Local device: mlx4_0
--------------------------------------------------------------------------
--------------------------------------------------------------------------
WARNING: There is at least one non-excluded one OpenFabrics device found,
but there are no active ports detected (or Open MPI was unable to use
them).  This is most certainly not what you wanted.  Check your
cables, subnet manager configuration, etc.  The openib BTL will be
ignored for this job.

  Local host: node01
--------------------------------------------------------------------------
[node01:1441597] 10 more processes have sent help message help-mpi-btl-openib.txt
    / error in device init
[node01:1441597] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help
```

```
    / error messages
run=1 time_sec=2.692526 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=2.689227 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=2.689532 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=12 runs=3 avg_time_sec=2.690428

Test III: Dual Socket (24 ranks)
---
Run 1:
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.

  Local host:   node01
  Local device: mlx4_0
--------------------------------------------------------------------------
--------------------------------------------------------------------------
WARNING: There is at least one non-excluded one OpenFabrics device found,
but there are no active ports detected (or Open MPI was unable to use
them).  This is most certainly not what you wanted.  Check your
cables, subnet manager configuration, etc.  The openib BTL will be
ignored for this job.

  Local host: node01
--------------------------------------------------------------------------
[node01:1441674] 22 more processes have sent help message help-mpi-btl-openib.txt
    / error in device init
[node01:1441674] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help
     / error messages
run=1 time_sec=1.760056 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=1.753550 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=1.760807 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=24 runs=3 avg_time_sec=1.758138
Run 2:
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.

  Local host:   node01
  Local device: mlx4_0
--------------------------------------------------------------------------
[node01:1441903] 23 more processes have sent help message help-mpi-btl-openib.txt
    / error in device init
[node01:1441903] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help
     / error messages
run=1 time_sec=1.756009 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=1.757776 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=1.756961 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=24 runs=3 avg_time_sec=1.756915
Run 3:
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.

  Local host:   node01
  Local device: mlx4_0
--------------------------------------------------------------------------
[node01:1442026] 23 more processes have sent help message help-mpi-btl-openib.txt
    / error in device init
[node01:1442026] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help
     / error messages
run=1 time_sec=1.758870 sorted_local=yes sorted_boundaries=yes
```

```
run=2 time_sec=1.756107 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=1.751988 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=24 runs=3 avg_time_sec=1.755655

Test IV: Distributed (48 ranks / 2 nodes)
---
Run 1:
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.

  Local host:   node01
  Local device: mlx4_0
--------------------------------------------------------------------------
--------------------------------------------------------------------------
WARNING: There is at least one non-excluded one OpenFabrics device found,
but there are no active ports detected (or Open MPI was unable to use
them).  This is most certainly not what you wanted.  Check your
cables, subnet manager configuration, etc.  The openib BTL will be
ignored for this job.

  Local host: node01
--------------------------------------------------------------------------
[node01:1442202] 41 more processes have sent help message help-mpi-btl-openib.txt
    / error in device init
[node01:1442202] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help
     / error messages
[node01:1442202] 5 more processes have sent help message help-mpi-btl-openib.txt /
     no active ports found
run=1 time_sec=1.515749 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=1.386695 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=1.350523 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=48 runs=3 avg_time_sec=1.417656
Run 2:
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.

  Local host:   node02
  Local device: mlx4_0
--------------------------------------------------------------------------
--------------------------------------------------------------------------
WARNING: There is at least one non-excluded one OpenFabrics device found,
but there are no active ports detected (or Open MPI was unable to use
them).  This is most certainly not what you wanted.  Check your
cables, subnet manager configuration, etc.  The openib BTL will be
ignored for this job.

  Local host: node02
--------------------------------------------------------------------------
[node01:1442319] 46 more processes have sent help message help-mpi-btl-openib.txt
    / error in device init
[node01:1442319] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help
     / error messages
run=1 time_sec=1.530647 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=1.353101 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=1.351100 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=48 runs=3 avg_time_sec=1.411616
Run 3:
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.
```

```
215
216   Local host:   node01
217   Local device: mlx4_0
218 --------------------------------------------------------------------------
219 --------------------------------------------------------------------------
220 WARNING: There is at least one non-excluded one OpenFabrics device found,
221 but there are no active ports detected (or Open MPI was unable to use
222 them).  This is most certainly not what you wanted.  Check your
223 cables, subnet manager configuration, etc.  The openib BTL will be
224 ignored for this job.
225
226   Local host: node01
227 --------------------------------------------------------------------------
228 [node01:1442440] 44 more processes have sent help message help-mpi-btl-openib.txt
      / error in device init
229 [node01:1442440] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help
      / error messages
230 [node01:1442440] 2 more processes have sent help message help-mpi-btl-openib.txt /
      no active ports found
231 run=1 time_sec=1.527339 sorted_local=yes sorted_boundaries=yes
232 run=2 time_sec=1.383580 sorted_local=yes sorted_boundaries=yes
233 run=3 time_sec=1.346258 sorted_local=yes sorted_boundaries=yes
234 N=268435456 ranks=48 runs=3 avg_time_sec=1.419059
235
236 Test V: Standard Test (96 ranks / 4 nodes)
237 ---
238 Run 1:
239 --------------------------------------------------------------------------
240 WARNING: There was an error initializing an OpenFabrics device.
241
242   Local host:   node01
243   Local device: mlx4_0
244 --------------------------------------------------------------------------
245 --------------------------------------------------------------------------
246 WARNING: There is at least one non-excluded one OpenFabrics device found,
247 but there are no active ports detected (or Open MPI was unable to use
248 them).  This is most certainly not what you wanted.  Check your
249 cables, subnet manager configuration, etc.  The openib BTL will be
250 ignored for this job.
251
252   Local host: node01
253 --------------------------------------------------------------------------
254 [node01:1442621] 88 more processes have sent help message help-mpi-btl-openib.txt
      / error in device init
255 [node01:1442621] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help
      / error messages
256 [node01:1442621] 6 more processes have sent help message help-mpi-btl-openib.txt /
      no active ports found
257 run=1 time_sec=1.360452 sorted_local=yes sorted_boundaries=yes
258 run=2 time_sec=0.960280 sorted_local=yes sorted_boundaries=yes
259 run=3 time_sec=0.975263 sorted_local=yes sorted_boundaries=yes
260 N=268435456 ranks=96 runs=3 avg_time_sec=1.098665
261 Run 2:
262 --------------------------------------------------------------------------
263 WARNING: There was an error initializing an OpenFabrics device.
264
265   Local host:   node02
266   Local device: mlx4_0
267 --------------------------------------------------------------------------
```

```
--------------------------------------------------------------------------
WARNING: There is at least one non-excluded one OpenFabrics device found,
but there are no active ports detected (or Open MPI was unable to use
them).  This is most certainly not what you wanted.  Check your
cables, subnet manager configuration, etc.  The openib BTL will be
ignored for this job.

  Local host: node02
--------------------------------------------------------------------------
[node01:1442746] 90 more processes have sent help message help-mpi-btl-openib.txt / error in device init
[node01:1442746] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help / error messages
[node01:1442746] 4 more processes have sent help message help-mpi-btl-openib.txt / no active ports found
run=1 time_sec=1.375364 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=0.961798 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=0.960480 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=96 runs=3 avg_time_sec=1.099214
Run 3:
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.

  Local host:   node03
  Local device: mlx4_0
--------------------------------------------------------------------------
--------------------------------------------------------------------------
WARNING: There is at least one non-excluded one OpenFabrics device found,
but there are no active ports detected (or Open MPI was unable to use
them).  This is most certainly not what you wanted.  Check your
cables, subnet manager configuration, etc.  The openib BTL will be
ignored for this job.

  Local host: node02
--------------------------------------------------------------------------
[node01:1442860] 92 more processes have sent help message help-mpi-btl-openib.txt / error in device init
[node01:1442860] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help / error messages
[node01:1442860] 2 more processes have sent help message help-mpi-btl-openib.txt / no active ports found
run=1 time_sec=1.362881 sorted_local=yes sorted_boundaries=yes
run=2 time_sec=1.018743 sorted_local=yes sorted_boundaries=yes
run=3 time_sec=0.964730 sorted_local=yes sorted_boundaries=yes
N=268435456 ranks=96 runs=3 avg_time_sec=1.115452

================================================================================
Benchmarking Complete
Date: Mon Apr 20 19:10:40 EDT 2026
================================================================================

Performance Metrics to Calculate:
1. Speedup (S_p) = T_serial / T_p
2. Efficiency (E_p) = (S_p / p) * 100%
   where p = number of ranks, T_serial = Test I time, T_p = execution time with p ranks

Results saved in output file: radix_results_$SLURM_JOB_ID.txt
```